Basic Data Structures: Dynamic Arrays and Amortized Analysis

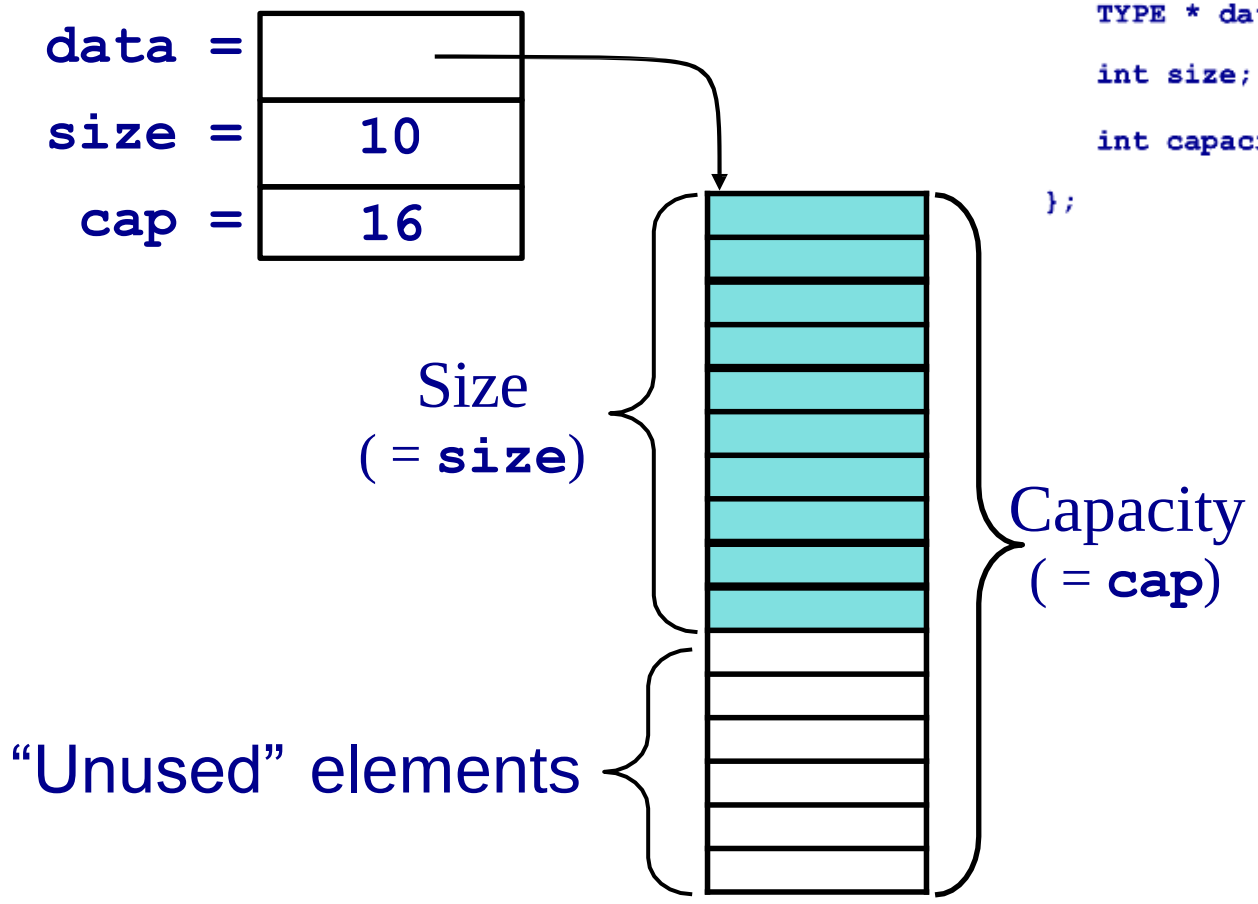
Data Structures

Data Structures and Algorithms

Outline

1. Dynamic Arrays
2. Amortized Analysis-Aggregate Method

Dynamic Array




```
struct dyArr {
   TYPE * data;   /* Pointer to data array */
   int size;      /* Number of elements */
   int capacity; /* Capacity of array */
};
```

Size and Capacity

- Size:
  - Current number of elements
  - Managed by an internal data value
- Capacity:
  - Number of elements that a dynamic array can hold before it must resize

Adding an Element

- Increment the size

- Put the new value at the end of the dynamic array

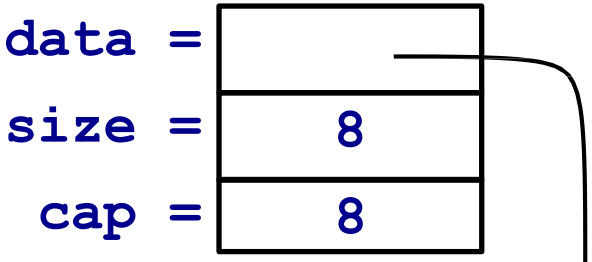
Adding an Element

- What happens when size == capacity?

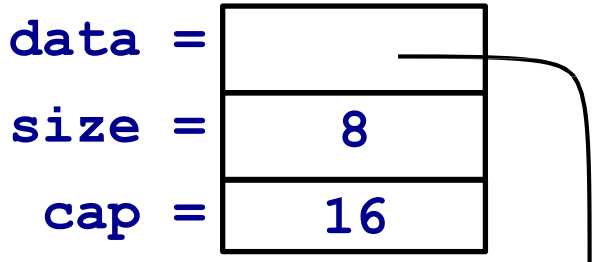
- Must:
  - reallocate new space
  - copy all data values to the new space
  - hide these details from the user

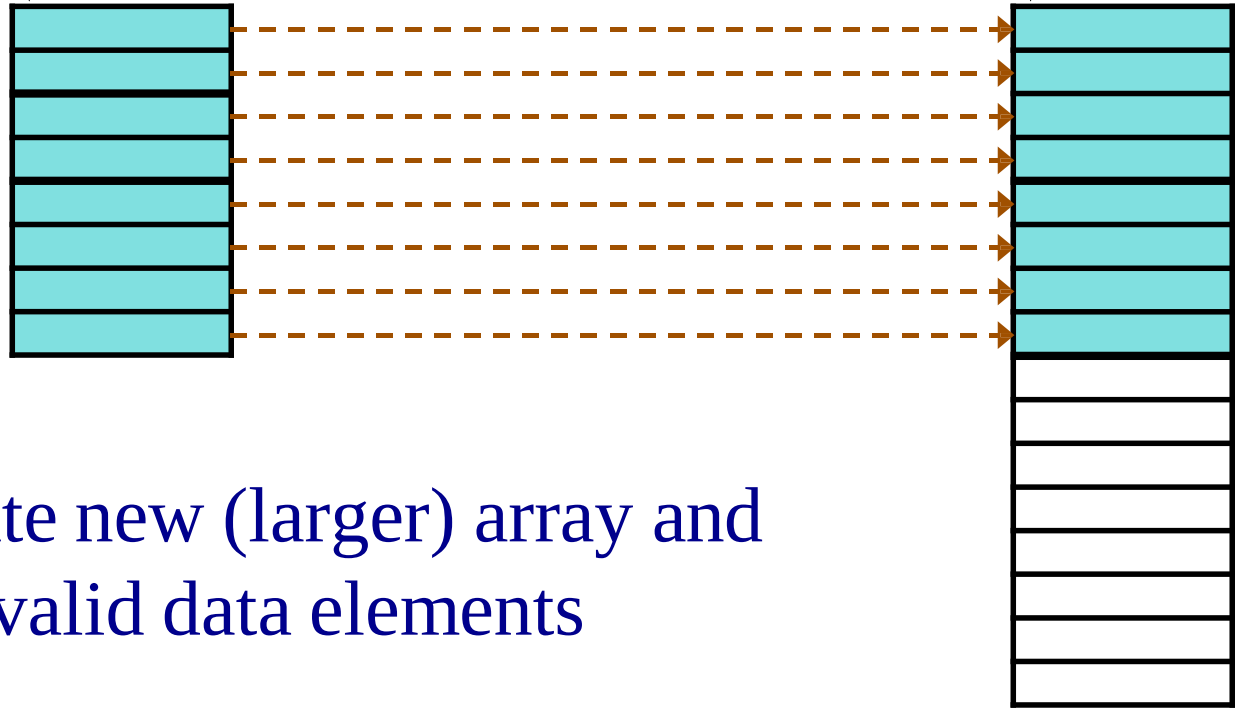
Reallocate and Copy

Before reallocation:



After reallocation:

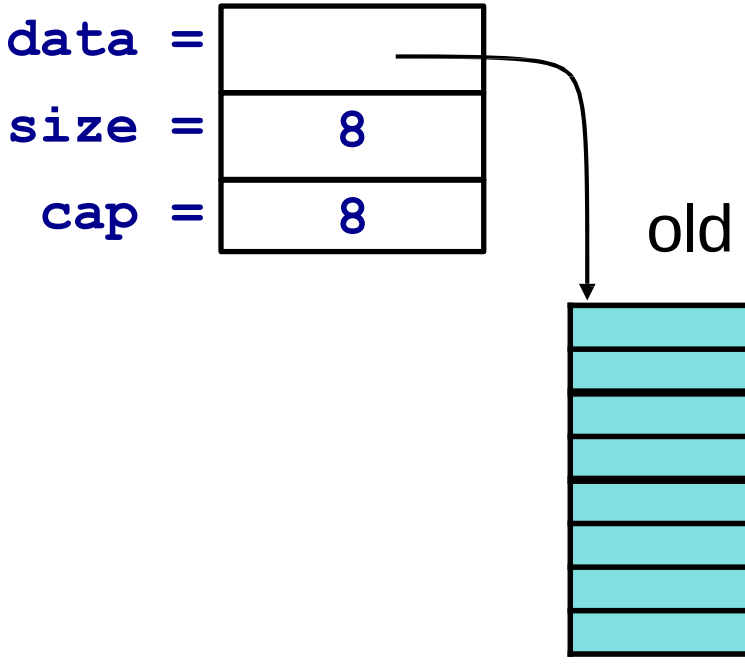




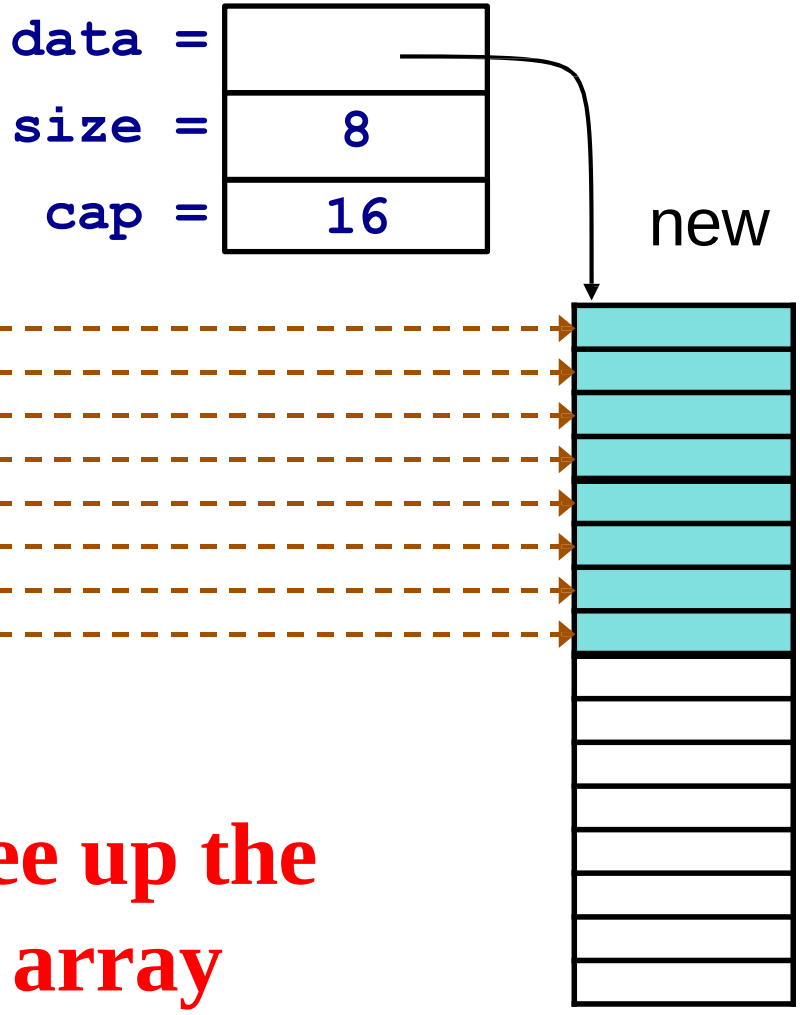
Must allocate new (larger) array and copy valid data elements

Reallocate and Copy

Before reallocation:



After reallocation:



DO NOT forget to free up the memory of the old array

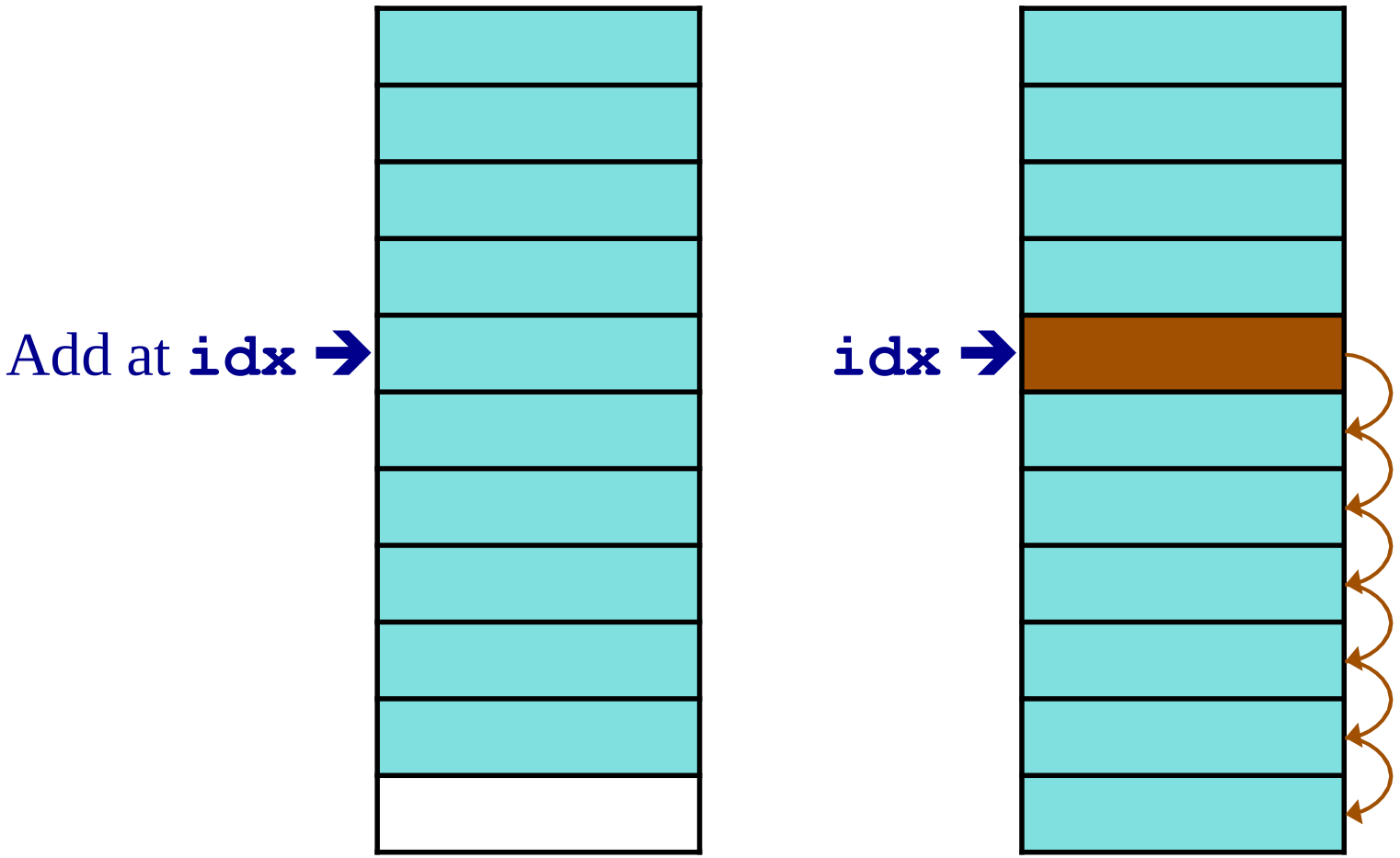
Inserting an Element in the Middle

- May also require reallocation
  - When?

- Requires that some elements be moved up to make space for the new one

Inserting an Element

Loop from **THE END backward** while copying

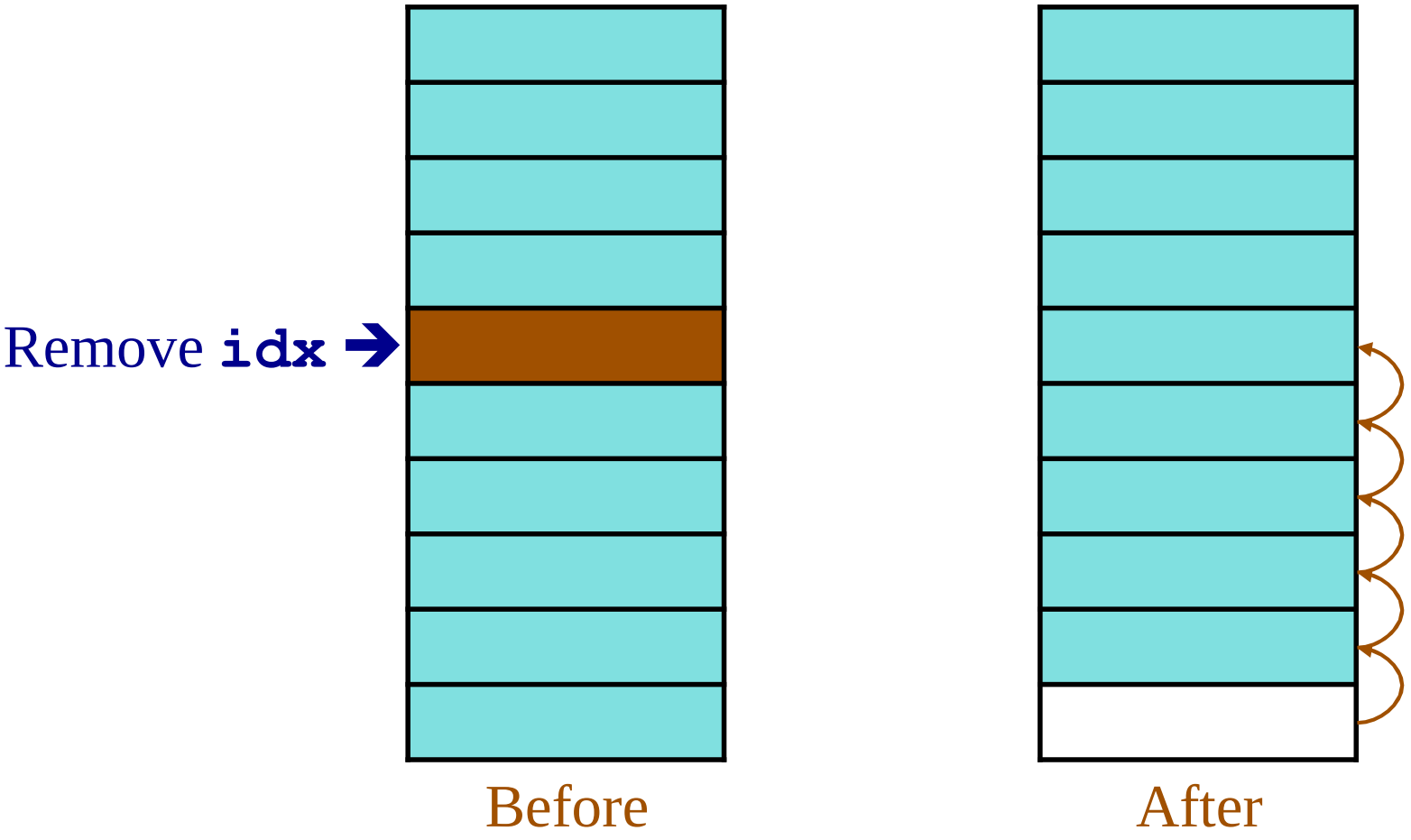


Inserting an Element -- Complexity

O(*n*) in the worst case

Removing an Element

- Remove also requires looping.
- Loop from **`idx`** **forward** while copying



Removing an Element -- Complexity

O(n) worst case

Interface View of Dynamic Arrays

Interface file: **dynArr.h**

```
struct dyArr {
    TYPE * data;   /* Pointer to data array */
    int size;      /* Number of elements */
    int capacity; /* Capacity of array */
};

/* Rest of dynarr.h on next slide */
```

Interface (continued)

```
/* function prototypes */

void initDynArr (struct dyArr *da, int cap);

void freeDynArr (struct dyArr *da);

void addDynArr (struct dyArr *da, TYPE val);

TYPE getDynArr (struct dyArr *da, int idx);

void putDynArr (struct dyArr *da, int idx, TYPE val);

int sizeDynArr (struct dyArr *da);

void _dyArrDoubleCapacity (struct dyArray * da);
```

Implementation View of Dynamic Arrays

initDynArr -- Initialization

```
   /* Allocate memory to data array */

void initDynArr (struct dyArr *da, int cap){
      assert (cap >= 0);
      da->capacity = cap;
      da->size = 0;
      da->data = (TYPE *)
                malloc(da->capacity * sizeof(TYPE));
      assert (da->data != 0); /* check the status */
}
```

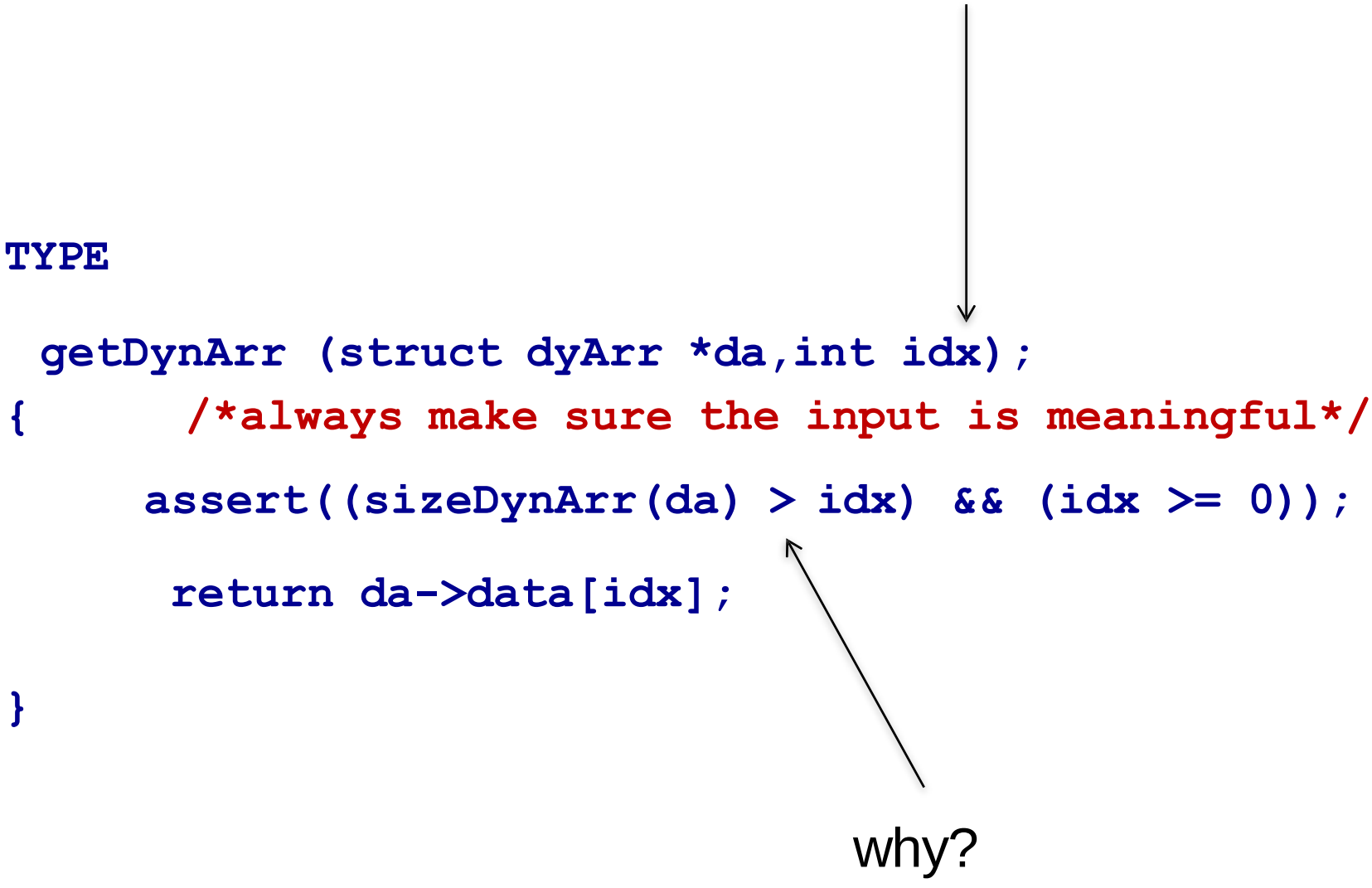
freeDynArr -- Clean-up

```
void freeDynArr (struct dyArr * da)
{
        assert (da != 0);
        free (da->data); /*free entire array*/
        da->capacity = 0;
        da->size = 0;
}
```

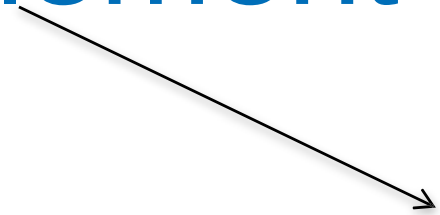
Size

```
int sizeDynArr (struct dyArr * da){
    return da->size;
}
```

Get the Value at a Given Position



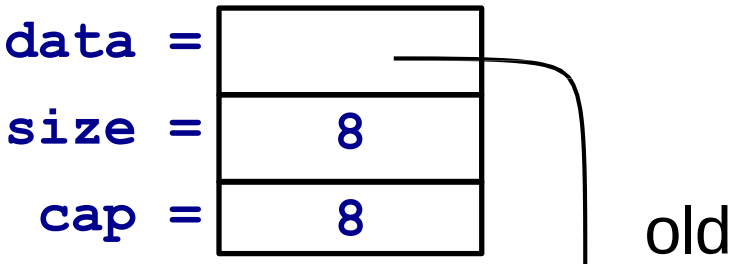
Add a New Element



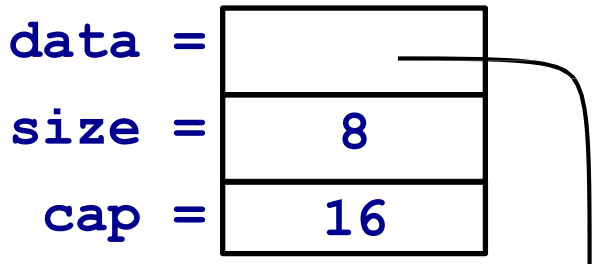
```
void addDynArr (struct dyArr * da, TYPE val){
        /*make sure there is enough capacity*/
        if (da->size >= da->capacity)
                        _dyArrDoubleCapacity(da);
        da->data[da->size] = val;
        da->size++; /*must increase the size*/
}
```

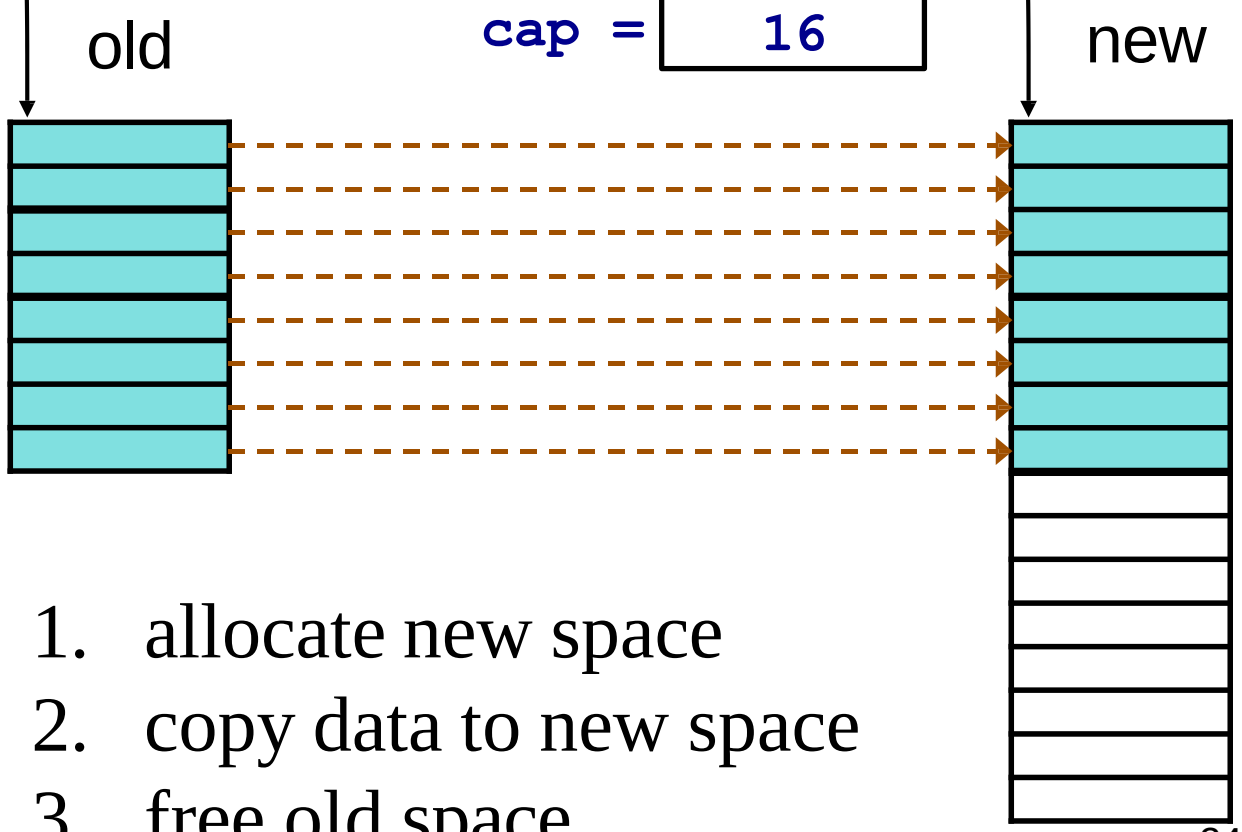
Double the Capacity

Before reallocation:



After reallocation:





MUST:

1. allocate new space
2. copy data to new space
3. free old space

Double the Capacity

```
void _dyArrDoubleCapacity (struct dyArray * da) {
      TYPE * oldbuffer = da->data; /*memorize old*/
      int oldsize = da->size;
      /*allocate new memory*/
      initDynArr (da, 2 * da->capacity);
      for (int i = 0; i < oldsize; i++) /*copy old*/
            da->data[i] = oldbuffer[i];
      da->size = oldsize;
      free(oldbuffer); /*free old memory*/
}
```

Runtimes

`getDynArr`	$O(1)$
`addDynArr`	$O(n)$
`dyArrDoubleCapacity`	$O(n)$
`sizeDynArr`	$O(1)$

Outline

1. Dynamic Arrays
2. Amortized Analysis-Aggregate Method

Dynamic Array

We only resize every so often.

Many $O(1)$ operations are followed by an

$O(n)$ operations.

What is the total cost of inserting many elements?

Definition

Amortized cost: Given a sequence of n operations, the amortized cost is:

$$\frac{\text{Cost}(n \text{ operations})}{n}$$

Aggregate Method

Dynamic array: n calls to addDynArr

Let c_i = cost of i'th insertion.

$$c_i = 1 + \begin{cases} i-1 & \text{if } i-1 \text{ is a power of 2} \\ 0 & \text{otherwise} \end{cases}$$

$$\frac{\sum_{i=1}^{n} c_i}{n} = \frac{n + \sum_{j=1}^{\lfloor \log_2(n-1) \rfloor} 2^j}{n} = \frac{O(n)}{n}$$

, $\sum_{j=1}^{\lfloor \log_2(n-1) \rfloor} 2^j = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 2.$

Note that $2^{\lfloor \log_2(n-1) \rfloor + 1} = 2 \cdot 2^{\lfloor \log_2(n-1) \rfloor} \leq 2 \cdot (n-1)$, so the total sum is at most $2(n-1) - 2 = O(n)$.

Question

For a dynamic array calculate amortized cost of an operation in the context of a sequence of operations. Please choose the growth factor = 2.

Outline

1. Dynamic Arrays
2. Amortized Analysis-Aggregate Method

Alternatives to Doubling the Array Size

We could use some different growth factor (1.5, 2.5, etc.).

Could we use a constant amount?

Cannot Use Constant Amount

If we expand by 10 each time, then:

Let c_i = cost of i'th insertion.

$$c_i = 1 + \begin{cases} i-1 & \text{if } i-1 \text{ is a multiple of 10} \\ 0 & \text{otherwise} \end{cases}$$

$$\frac{\sum_{i=1}^{n} c_i}{n} = \frac{n + \sum_{j=1}^{(n-1)/10} 10j}{n} = \frac{n + 10\sum_{j=1}^{(n-1)/10} j}{n}$$

$$= \frac{n + 10O(n^2)}{n} = \frac{O(n^2)}{n} = O(n)$$